

Technische Universität Clausthal

Department of Informatics

Beyond Pairwise GUI Quality Judgments

Multi-Dimensional Evaluation with Modern Multimodal LLMs

Master's Thesis

Xinyang Xie

May 18, 2026

Contents

1	Introduction	1
2	Background and Related Work	2
2.1	Research Gap	2
3	Research Questions and Methodology	5
3.1	Research Questions	5
3.2	Overall Study Design	7
3.3	Track A: Screenshot-based Reproduction	7
3.4	Track B: Requirement-driven Generated Interfaces	8
3.4.1	Requirement Sources	8
3.4.2	Interface Generation	8
3.4.3	Rendering Pipeline	9
3.5	Static Evaluation Methodology	9
3.5.1	Model Selection	9
3.5.2	Prompting Conditions	9
3.5.3	Reliability Conditions	9
3.6	Dynamic Evaluation Methodology	10
3.6.1	Approach	10
3.6.2	Task Derivation	10
3.6.3	Failure Taxonomy	10
3.7	Metrics	11
3.8	Research Question – Method Map	12

Chapter 1

Introduction

This chapter will introduce the motivation, research context, and contribution of the thesis. It is intentionally kept as a short placeholder so that the following draft chapters keep their intended numbering while the full introduction is developed.

Chapter 2

Background and Related Work

2.1 Research Gap

The preceding sections have reviewed GUI quality assessment, LLM-generated interfaces, LLM-as-a-judge methodology, and multimodal web agents. Table 2.1 places the most relevant prior works along seven dimensions that together characterise a comprehensive GUI quality evaluation framework. Taken together, three observations motivate the present thesis.

No prior work provides a multi-metric leaderboard for LLM-generated GUIs. UIClip [?] demonstrated that vision-language models can perform pairwise UI comparison, but it relies on models that predate the current generation of multimodal LLMs and limits evaluation to a binary preference. The LLM-as-a-judge literature [?] has established systematic judge evaluation for text, yet this methodology has not been transferred into a reusable benchmark for comparing the quality of generated graphical user interfaces across static and dynamic metrics.

Requirement fidelity and rubric-based assessment are largely absent from GUI evaluation. Design2Code [?] evaluates how faithfully generated code reproduces a visual reference, but does not assess richer quality dimensions such as visual hierarchy, information clarity, or perceived usability. The Designer Feedback study [?] found that simple pairwise rankings among evaluators agreed at only around 50%, suggesting that coarse binary judgements are insufficient for reliable GUI assessment. A structured rubric grounded in established usability theory [?] can provide finer-grained and more reproducible quality signals.

Static visual metrics have rarely been related to dynamic interaction outcomes. VisualWebArena [?] and WebArena [?] evaluate multimodal agents on goal-directed web tasks, but do not assess the GUI quality of the interfaces themselves.

How static quality scores relate to task completion, interaction feedback, and functional behaviour remains an open empirical question. Answering this question requires a benchmark that keeps generated interfaces executable so that static and dynamic evaluations can be performed on the same set of items.

The present thesis addresses all three gaps simultaneously by combining a two-track benchmark, a rubric-based human annotation protocol, a systematic evaluation methodology with bias and reliability analysis, and a static-plus-dynamic leaderboard for comparing the quality of LLM-generated GUIs.

Table 2.1: Comparison of related work across seven evaluation dimensions. $\checkmark$ = fully addressed; $\triangle$ = partially addressed; $\times$ = not addressed; — = not applicable to this line of work.

Work	Multi-dimensional rubric	Generated GUI quality	Bias and reliability analysis	Human reference labels	Requirement fidelity	Dynamic interaction eval	Benchmark artifact
UIClip [?]]	×	×	×	✓	×	×	×
Designer Feedback [?]]	×	✓	×	✓	△	×	×
Design2Code [?]]	△	✓	×	✓	✓	×	✓
MT-Bench / Chatbot Arena [?]]	△	—	△	✓	—	—	△
LLM-as-a-Judge survey [?]]	✓	—	✓	—	—	—	×
Position Bias Study [?]]	×	—	✓	✓	—	—	×
WebArena [?]]	×	△	×	✓	△	✓	✓
VisualWebArena [?]]	×	✓	×	✓	△	✓	✓
This thesis	✓	✓	✓	✓	✓	✓	✓

Chapter 3

Research Questions and Methodology

This chapter translates the motivation established in Chapter 2 into four precise research questions and describes the experimental design through which each question is answered. Section 3.1 states the research questions. Section 3.2 outlines the overall study design. Sections 3.3–3.6 detail the two dataset tracks and the static and dynamic evaluation methodologies. Section 3.7 defines all evaluation metrics. Section 3.8 maps each research question to the corresponding dataset, tasks, metrics, and expected outputs.

3.1 Research Questions

The central concern of this thesis is how the quality of LLM-generated graphical user interfaces can be evaluated with a reusable multi-metric benchmark and leaderboard. The evaluation combines static visual and requirement-based metrics with dynamic functionality signals, while using human annotations and LLM-based evaluators as measurement instruments rather than as the main object of the leaderboard. Four research questions operationalise this concern.

RQ1 – UIClip-style baseline and pairwise evaluation limits.

What does a reduced UIClip-style reproduction reveal about modern multi-modal models in pairwise GUI-quality judgement, and which methodological controls are needed before such judgements can be used inside a GUI evaluation benchmark?

This question targets the reproduction aspect of the study (Track A, Section 3.3). It uses the UIClip dataset as an anchor for pairwise screenshot evaluation, but its role is methodological: to establish a baseline, identify limitations such as position bias,

and motivate the move from single binary preferences toward richer multi-metric GUI evaluation.

RQ2 – Static multi-metric quality of LLM-generated GUIs.

How can LLM-generated GUIs be evaluated with static multi-dimensional metrics, and how well do automated or LLM-based scores align with human ratings across visual structure, information clarity, and requirement fidelity?

This question is centred on Track B (Section 3.4), where different generator models produce interfaces from the same requirements. It examines whether the static portion of the leaderboard can provide interpretable per-dimension quality signals rather than only a single aggregate score.

RQ3 – Evaluation method reliability and cost.

Among the tested judging strategies – zero-shot prompting, rubric-guided prompting, order-swapped pairwise comparisons, and repeated sampling – which configurations provide the most reliable and cost-effective measurement signals for the leaderboard?

This question is answered across both tracks but remains a methodological supporting question. It draws on the LLM-as-a-judge literature [? ? ?] to select strategies known to affect judge reliability, and applies them systematically to the GUI evaluation setting. Bias analysis is therefore treated as a control and reporting requirement rather than as the thesis’s central contribution.

RQ4 – Static and dynamic GUI quality signals.

For the same set of executable Track B interfaces, how are static quality scores related to dynamic functionality and task-validation outcomes, and where do the two kinds of metrics complement or diverge from each other?

This is the core empirical question of the thesis. Prior work has evaluated GUI quality from screenshots or evaluated agents on web tasks, but the two lines of research have not been connected on the same set of interfaces. RQ4 addresses this gap by pairing a static rubric score vector with a dynamic plan-success rate for every Track B item and analysing both correlations and mismatch cases. The goal is not to assume that static and dynamic metrics should be identical, but to understand what each signal captures about the quality of generated GUIs.

3.2 Overall Study Design

Figure 3.1 illustrates the end-to-end study design. The evaluation is organised around two complementary dataset tracks.

Track A consists of GUI screenshot pairs drawn from the UIClip dataset. Its primary function is reproduction and anchoring: by running modern multimodal LLMs on the same pairwise task that UIClip used, Track A enables a direct numerical comparison that establishes whether contemporary models represent a measurable advance. Track A items undergo static evaluation only.

Track B consists of graphical user interfaces generated from natural language requirements. Every Track B interface is kept in executable form so that it can be assessed both statically – through rubric-based scoring – and dynamically, through LLM-derived interaction plans that are validated against the rendered DOM. The key design principle is that the same interface receives both a static rubric score vector and a dynamic plan-success measurement, forming the paired data needed to answer RQ4.

The primary leaderboard unit is the generated GUI submission, aggregated by generator model and requirement set. Evaluator models, prompts, and human annotations are part of the evaluation protocol used to compute the metrics; they are not the main entities being ranked.

Human annotations provide the reference labels for both tracks. Track A reuses the preference labels published with the UIClip dataset, so no new annotation is required for pairwise tasks on that track. Track B requires fresh human rubric scores, collected according to the annotation protocol described in the benchmark chapter.

All experiments are documented with exact model identifiers, API version dates, prompt text, random seeds, and output files to support reproducibility.

3.3 Track A: Screenshot-based Reproduction

Track A draws on the publicly available UIClip dataset [?], which contains pairs of GUI screenshots each labelled with a human preference indicating which interface is visually superior. A random sample of 50 to 100 pairs is selected, stratified to cover the range of quality contrasts present in the original dataset.

No new human annotation is collected for Track A. The original UIClip preference labels serve as ground-truth reference, and agreement between each evaluator model and those labels constitutes the alignment measure. This design makes Track A lightweight and directly comparable to the UIClip publication while keeping the main thesis contribution – a static and dynamic leaderboard for LLM-generated GUIs – concentrated in Track B.

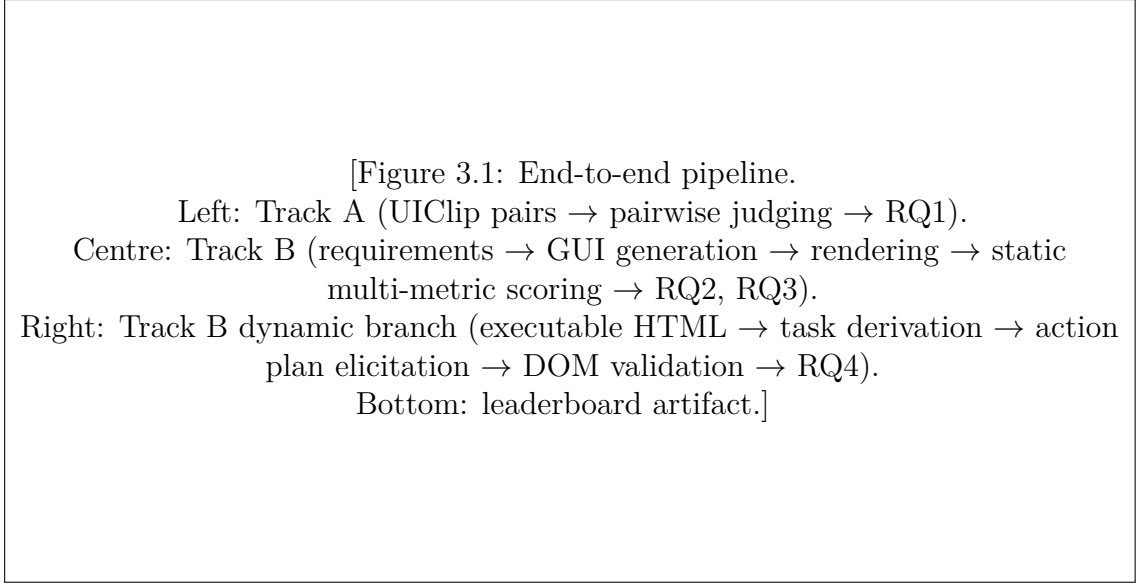


Figure 3.1: End-to-end evaluation pipeline. Track A supports reproduction (RQ1). Track B supports static multi-metric assessment (RQ2, RQ3) and dynamic plan validation (RQ4).

3.4 Track B: Requirement-driven Generated Interfaces

3.4.1 Requirement Sources

Track B requirements are collected from two sources. The primary source is the Design2Code-HARD subset [?], which contains 80 challenging real-world webpages. For each selected page, the corresponding HTML reference is used to derive a concise natural language requirement that describes the intended layout, content, and functionality without specifying implementation details. Approximately 50 to 60 pages are selected after filtering out items whose interactive functionality cannot be meaningfully assessed through DOM inspection alone.

A secondary set of 5 to 10 requirements is written manually to cover scenarios underrepresented in Design2Code-HARD, specifically form-heavy interfaces and simplified mobile-style layouts. These types support more tractable dynamic task derivation (Section 3.6) and reduce reliance on a single source.

3.4.2 Interface Generation

Each requirement is submitted to two or three generator models, producing a set of generated HTML implementations per requirement. The generator models are drawn from the same pool as the evaluator models (Section 3.5) where feasible, but generator identity and evaluator identity are recorded separately. The leaderboard compares the

quality of the generated interfaces; evaluator behaviour is analysed only as part of the evaluation methodology.

3.4.3 Rendering Pipeline

Generated HTML files are rendered using a headless browser (Playwright) at a fixed viewport resolution. For each item, the pipeline saves a PNG screenshot and a serialised DOM snapshot. Items whose rendering produces a blank page or a JavaScript console error that prevents visible content from loading are excluded and replaced. The rendered screenshot and the DOM snapshot together constitute the input to both the static and dynamic evaluation modules.

3.5 Static Evaluation Methodology

3.5.1 Model Selection

Four evaluator models are tested to span a range of capability levels. Two closed-source multimodal models represent the current state of the art; one open-weight multimodal model is included to allow reproducibility without API access; and one smaller or older multimodal model serves as a weaker baseline that makes the relationship between model capability and evaluator quality visible. Exact model identifiers and access dates are recorded in the reproducibility appendix.

3.5.2 Prompting Conditions

Two prompting conditions are applied to every model.

Zero-shot prompting presents the model with the interface screenshot and a minimal instruction to indicate which of two interfaces is better (pairwise) or to score the interface on the rubric dimensions. This condition establishes a baseline.

Rubric-guided prompting supplies the full rubric with dimension definitions and per-level score anchors, and requires the model to return a structured JSON response containing one integer score per dimension and a one-sentence rationale. This condition tests whether explicit guidance improves alignment with human ratings.

3.5.3 Reliability Conditions

Two reliability conditions are applied on top of the prompting conditions.

Order-swapped pairwise comparison repeats each pairwise judgement with the two interface screenshots presented in the reverse order [?]. If a model labels interface A as better when it appears first but interface B as better when it appears first, the

judgement is classified as position-biased. The position-bias rate is computed per model and per prompting condition.

Repeated sampling submits each prompt three times with a non-zero sampling temperature. The self-consistency rate measures the proportion of items on which all three runs produce the same judgement.

3.6 Dynamic Evaluation Methodology

3.6.1 Approach

A computer-use agent that executes browser actions introduces two confounded sources of noise: failures caused by the interface itself and failures caused by the agent’s own planning or grounding limitations. Disentangling these sources is difficult and adds substantial engineering overhead.

Instead, this thesis uses *action plan elicitation*: an evaluator model is given the rendered screenshot, the DOM snapshot, and a short task description derived from the original requirement, and is asked to produce an ordered action plan – a JSON list of steps, each specifying an action type, a CSS or ARIA target selector, and an optional value. A deterministic validation script then checks whether each step in the plan is feasible: whether the target element exists in the DOM, whether the selector resolves to a unique element, and whether a form-submission step reaches a plausible end state. This design isolates interface quality from agent execution variability and produces a binary feasibility verdict for each action step, giving a plan-success rate per item.

Full computer-use agent execution – where an agent autonomously interacts with the live rendered page – is left as a future extension.

3.6.2 Task Derivation

One to two tasks are derived from each Track B requirement. A task is a single imperative sentence with an unambiguous success criterion that can be checked by inspecting the DOM or a screenshot of the expected end state. Example tasks include filling and submitting a contact form, selecting an item from a navigation menu, or entering a search query. Tasks are written by the author and cross-checked by one independent reviewer to confirm clarity and feasibility.

3.6.3 Failure Taxonomy

Action plan failures are classified into four categories to support interpretable analysis of RQ4.

F1 – Interface failure: the target element is absent from the DOM, the layout prevents meaningful interaction, or a required interactive component is not implemented.

F2 – Plan failure: the element exists but the model’s selector does not resolve correctly, or the action sequence does not reach the target state despite the interface being technically functional.

F3 – Ambiguity: the task description admits multiple valid interpretations and the model’s plan addresses a different valid interpretation than the one used by the validator.

F4 – Tool failure: the validation script itself produces an error unrelated to the interface or the plan.

Cases in category F3 and F4 are reviewed manually and excluded from the RQ4 correlation analysis if they cannot be resolved.

3.7 Metrics

Human-model agreement (pairwise). Cohen’s κ measures agreement between an evaluator model’s pairwise preference and the human majority label, correcting for chance.

Human-model correlation (rubric). Spearman’s ρ measures rank correlation between model rubric scores and human rubric scores, computed per rubric dimension and reported both individually and averaged.

Position-bias rate. The proportion of pairwise items on which the model’s preference reverses between the original and order-swapped presentation.

Self-consistency rate. The proportion of items on which all three repeated-sampling runs produce the same judgement or score.

Plan-success rate. The proportion of action steps in a plan that pass all three feasibility checks (existence, unique resolution, state transition). Averaged across steps within an item to obtain an item-level score.

Static-dynamic relationship. Spearman’s ρ between the item-level static rubric score (aggregate and per-dimension) and the item-level plan-success rate, computed across all Track B items. In addition, mismatch cases are reported qualitatively, because a visually strong interface can still fail a task and a visually plain interface can still be functionally successful.

Table 3.1: Research question – method mapping.

RQ	Question (short)	Track	Task type	Primary metrics
1	UIClip-style pairwise baseline and evaluation limits	Track A	Pairwise judging	Accuracy vs UIClip-derived labels; position-bias rate
2	Static multi-metric quality of generated GUIs	Track B	Rubric scoring	Human-model Spearman ρ per dimension; leaderboard scores
3	Reliable and cost-effective evaluation methodology	Track A + B	Pairwise + rubric	κ ; ρ ; position-bias rate; self-consistency rate; API cost
4	Relationship between static and dynamic quality signals	Track B	Rubric + action plan validation	Spearman ρ (static-dynamic); plan-success rate; mismatch categories

API cost. Total token cost in USD per model per condition, computed from provider pricing at the time of the experiment and used to characterise the alignment-cost trade-off addressed in RQ3.

3.8 Research Question – Method Map

Table 3.1 summarises the relationship between each research question, the dataset track and task type used to answer it, the primary metrics, and the results section that reports the findings.